

Numeryczna symulacja równania Cahna–Hilliarda 2D

Rozkład spinodalny i koalescencja faz metodą spektralną

Projekt z Podstaw Fizyki 3

Wydział Fizyki, Politechnika Warszawska

6 czerwca 2026

Kod źródłowy: <https://github.com/sosna77/cahn-hilliard>

Spis treści

1	Wstęp	2
2	Wyprowadzenie teoretyczne	2
2.1	Funkcjonał energii swobodnej Ginzburga–Landaua	2
2.2	Potencjał chemiczny i prąd dyfuzji	3
2.3	Adimensjonalizacja	3
2.4	Energia swobodna w postaci kanonicznej	4
3	Metoda numeryczna – SISM	4
3.1	Motywacja metody spektralnej	4
3.2	Równanie C–H w przestrzeni Fouriera	4
3.3	Pół-niejawny schemat Eulera	5
3.4	Algorytm kroku czasowego	5
3.5	Reguła dealiasingowa 2/3	6
4	Implementacja	6
4.1	Struktura projektu	6
4.2	Parametry konfiguracyjne	6
4.3	Kluczowe fragmenty kodu	6
4.4	Obliczanie energii	8
4.5	Architektura wielowątkowa	8
4.6	Akceleracja z Numbą	8
5	Wyniki	8
5.1	Przypadek $m_0 = 0$: struktury labiryntowe	9
5.2	Przypadek $m_0 = +0.35$: krople mniejszościowe	9
5.3	Przypadek $m_0 = -0.35$: krople odwrotne	10
5.4	Weryfikacja zachowania energii	11
6	Podsumowanie	11

1. Wstęp

Równanie Cahn–Hilliarda (C–H), zaproponowane w 1958 roku przez Johna W. Cahn i Johna E. Hilliarda [1], opisuje ewolucję składu faz w układzie dwóch wzajemnie nierozpuszczalnych składników. W kontekście magnetycznym, którego dotyczy niniejszy projekt, pole $m(\mathbf{r}, t) \in [-1, +1]$ reprezentuje lokalną *polaryzację spinową* (magnetyzację) w modelu Isinga.

Dwa kluczowe zjawiska opisywane przez to równanie to:

- **Rozkład spinodalny** (*spinodal decomposition*) – spontaniczne rozdzielanie się jednorodnej mieszaniny na dwie fazy w wyniku niestabilności termodynamicznej; dominuje przy $m_0 \approx 0$, tworząc charakterystyczne struktury labiryntowe.
- **Koalescencja faz** (*coarsening, Ostwald ripening*) – długoczasowy wzrost domen fazowych, napędzany minimalizacją energii powierzchniowej (granicy faz).

Celem projektu była implementacja numerycznego solvera C–H w 2D z wykorzystaniem *pół-niejawnej metody spektralnej* (SISM) oraz kompilatora JIT (Numba), a następnie budowa interaktywnego interfejsu graficznego umożliwiającego obserwację ewolucji w czasie rzeczywistym.

2. Wyprowadzenie teoretyczne

2.1. Funkcjonał energii swobodnej Ginzburga–Landaua

Przyjmujemy pole porządku $m(\mathbf{r}, t)$, $m \in [-1, +1]$, gdzie ± 1 odpowiadają dwóm fazom („spin w górę” / „spin w dół”).

Energię swobodną układu opisuje funkcyjonał Ginzburga–Landaua:

$$\mathcal{F}[m] = \int \left(-\frac{a}{2} m^2 + \frac{b}{4} m^4 + \frac{\kappa}{2} |\nabla m|^2 \right) d\mathbf{r}, \quad (2.1)$$

gdzie:

- $a, b > 0$ – parametry potencjału (podwójne minimum przy $m = \pm \sqrt{a/b}$),
- $\kappa > 0$ – koszt energetyczny gradientu (napężenie powierzchniowe),
- człon $-\frac{a}{2} m^2 + \frac{b}{4} m^4$ to potencjał Landaua z dwoma minimami $m_0 = \pm \sqrt{a/b}$.

2.2. Potencjał chemiczny i prąd dyfuzji

Potencjał chemiczny (pochodna funkcjonalna energii) jest zdefiniowany jako:

$$\mu = \frac{\delta \mathcal{F}}{\delta m} = -am + bm^3 - \kappa \nabla^2 m. \quad (2.2)$$

Strumień dyfuzji jest proporcjonalny do gradientu potencjału:

$$\mathbf{J} = -M \nabla \mu, \quad (2.3)$$

gdzie $M > 0$ to *ruchliwość* (mobilność). Równanie ciągłości (zachowanie składu) daje:

$$\frac{\partial m}{\partial t} = -\nabla \cdot \mathbf{J} = M \nabla^2 (-am + bm^3 - \kappa \nabla^2 m). \quad (2.4)$$

Przyjmując $M = 1$ (jednostka czasu pochłania mobilność), otrzymujemy **równanie Cahn–Hilliarda**:

$$\boxed{\frac{\partial m}{\partial t} = \nabla^2 (-am + bm^3 - \kappa \nabla^2 m)}. \quad (2.5)$$

2.3. Adimensjonalizacja

Stany równowagi wynikają z warunku $\mu = 0$:

$$-am_0 + bm_0^3 = 0 \implies m_0 = \sqrt{\frac{a}{b}}. \quad (2.6)$$

Wprowadzamy bezwymiarowe zmienne ($\tilde{m} = m/m_0$, $\tilde{\mathbf{r}} = \mathbf{r}/L$, $\tilde{t} = t/\tau$):

$$\tilde{m} = \frac{m}{m_0}, \quad m_0 = \sqrt{\frac{a}{b}}, \quad (2.7)$$

$$\tilde{\mathbf{r}} = \frac{\mathbf{r}}{L}, \quad \tilde{\nabla}^2 = L^2 \nabla^2, \quad (2.8)$$

$$\tilde{t} = \frac{t}{\tau}, \quad \tau = \frac{L^2}{Ma}. \quad (2.9)$$

Wstawiając do równania (2.4) i upraszczając:

$$\frac{\partial \tilde{m}}{\partial \tilde{t}} = \tilde{\nabla}^2 (\tilde{m}^3 - \tilde{m} - \varepsilon^2 \tilde{\nabla}^2 \tilde{m}), \quad \varepsilon^2 = \frac{\kappa}{aL^2}. \quad (2.10)$$

Po pominięciu tyldy (wszystkie zmienne bezwymiarowe) ostateczna **kanoniczna postać równania Cahn–Hilliarda**:

$$\boxed{\frac{\partial m}{\partial t} = \nabla^2 (m^3 - m) - \varepsilon^2 \nabla^4 m}. \quad (2.11)$$

Parametr ε kontroluje szerokość granicy faz: dla siatki o kroku Δx przyjmujemy $\varepsilon = \alpha \Delta x$, gdzie $\alpha \geq 1.5$ zapewnia stabilność numeryczną i brak artefaktów aliasingu.

2.4. Energia swobodna w postaci kanonicznej

W bezwymiarowych zmiennych całkowita energia swobodna przyjmuje postać:

$$\mathcal{F}[m] = \int \left[\frac{1}{4} (m^2 - 1)^2 + \frac{\varepsilon^2}{2} |\nabla m|^2 \right] d\mathbf{r}, \quad (2.12)$$

przy czym $\mathcal{F} \geq 0$ i maleje monotonicznie w czasie, co stanowi kluczową weryfikację poprawności solvera.

Dyskretna aproksymacja 2D:

$$\mathcal{F} \approx \sum_{i,j} \left[\frac{1}{4} (m_{i,j}^2 - 1)^2 + \frac{\varepsilon^2}{2} \left(\left(\frac{\partial m}{\partial x} \right)_{i,j}^2 + \left(\frac{\partial m}{\partial y} \right)_{i,j}^2 \right) \right] \Delta x \Delta y. \quad (2.13)$$

3. Metoda numeryczna – SISIM

3.1. Motywacja metody spektralnej

Równanie (2.11) jest *szttywne* (stiff): liniowy operator $-\varepsilon^2 \nabla^4 m$ zawiera czwartą pochodną przestrzenną i narzuca bardzo małe kroki czasowe przy jawnej integracji ($\Delta t \sim \Delta x^4$). Przeniesienie tego członu do *niejawnej* części schematu pozwala używać kroków $\Delta t \sim \Delta x^2$.

Transformata Fouriera diagonalizuje operatory różniczkowe:

$$\mathcal{F}\{\nabla^2 f\} = -k^2 \hat{f}_{\mathbf{k}}, \quad (3.1)$$

$$\mathcal{F}\{\nabla^4 f\} = +k^4 \hat{f}_{\mathbf{k}}, \quad (3.2)$$

gdzie $k^2 = k_x^2 + k_y^2$ i $\hat{f}_{\mathbf{k}} = \mathcal{F}\{f\}_{\mathbf{k}}$.

3.2. Równanie C–H w przestrzeni Fouriera

Po transformacji (2.11):

$$\frac{\partial \hat{m}_{\mathbf{k}}}{\partial t} = -k^2 \mathcal{F}\{m^3 - m\}_{\mathbf{k}} - \varepsilon^2 k^4 \hat{m}_{\mathbf{k}}. \quad (3.3)$$

3.3. Pół-niejawny schemat Eulera

Człon nieliniowy $f = m^3 - m$ traktujemy jawnie (krok n), a liniowy sztywny człon $\varepsilon^2 k^4$ – niejawnie (krok $n + 1$). Dyskretyzacja czasowa metodą Eulera ($t_{n+1} = t_n + \Delta t$):

$$\frac{\hat{m}_{\mathbf{k}}^{n+1} - \hat{m}_{\mathbf{k}}^n}{\Delta t} = -k^2 \hat{f}_{\mathbf{k}}^n - \varepsilon^2 k^4 \hat{m}_{\mathbf{k}}^{n+1}. \quad (3.4)$$

Grupując wyrazy z $\hat{m}_{\mathbf{k}}^{n+1}$:

$$\boxed{\hat{m}_{\mathbf{k}}^{n+1} = \frac{\hat{m}_{\mathbf{k}}^n - \Delta t k^2 \hat{f}_{\mathbf{k}}^n}{1 + \Delta t \varepsilon^2 k^4}} \quad (3.5)$$

gdzie $\hat{f}_{\mathbf{k}}^n = \mathcal{F}\{(m^n)^3 - m^n\}_{\mathbf{k}}$.

Schemat (3.5) jest **algebraicznie jawny** – nie wymaga rozwiązywania układów liniowych – a jednocześnie stabilny dla dużych kroków czasowych dzięki implicitnemu traktowaniu sztywnego członu.

3.4. Algorytm kroku czasowego

Pełny algorytm dla jednego kroku Δt :

1. Oblicz potencjał nieliniowy w przestrzeni rzeczywistej:

$$f^n(\mathbf{r}) = (m^n(\mathbf{r}))^3 - m^n(\mathbf{r}).$$

2. Wykonaj transformaty Fouriera:

$$\hat{f}_{\mathbf{k}}^n = \text{FFT2}(f^n), \quad \hat{m}_{\mathbf{k}}^n = \text{FFT2}(m^n).$$

3. Zastosuj maskę *dealiasingową* (reguła 2/3):

$$\hat{f}_{\mathbf{k}}^n \leftarrow \hat{f}_{\mathbf{k}}^n \cdot \mathbf{1}[|\mathbf{k}| < \frac{2}{3}k_{\max}].$$

4. Zaktualizuj w przestrzeni Fouriera:

$$\hat{m}_{\mathbf{k}}^{n+1} = \frac{\hat{m}_{\mathbf{k}}^n - \Delta t k^2 \hat{f}_{\mathbf{k}}^n}{1 + \Delta t \varepsilon^2 k^4}.$$

5. Odwrotna transformata:

$$m^{n+1}(\mathbf{r}) = \text{Re}[\text{IFFT2}(\hat{m}_{\mathbf{k}}^{n+1})].$$

3.5. Reguła dealiasingowa 2/3

Człon nieliniowy m^3 generuje tryby Fouriera o trzykrotnie wyższych częstotliwościach niż m . Bez dealiasingu dochodzi do *zawijania* (aliasingu) – wysokoczęstotliwościowe tryby fałszywie trafiają do niskich. Reguła 2/3 usuwa wszystkie tryby z $|\mathbf{k}| \geq \frac{2}{3}k_{\max}$, eliminując błędy przy zachowaniu dokładności dla istotnych skali.

4. Implementacja

4.1. Struktura projektu

Plik	Opis
main.py	Punkt wejściowy; inicjalizuje aplikację Qt
config.py	Klasa <code>SimConfig</code> (dataclass) z parametrami symulacji
engine.py	Logika fizyczna: <code>SimState</code> , <code>Solver</code> , <code>CHWorker</code>
gui.py	Interfejs graficzny (PySide6 + pyqtgraph)

Tabela 1: Struktura plików projektu.

4.2. Parametry konfiguracyjne

Parametr	Wartość domyślna	Opis
N	512	Rozdzielczość siatki ($N \times N$); potęga 2 optymalna dla FFT
L	10.0	Fizyczny rozmiar domeny; $\Delta x = L/N$
Δt	5×10^{-4}	Krok czasowy
m_0	0.0	Średnia magnetyzacja (skład początkowy)
α	1.5	$\varepsilon = \alpha \Delta x$; minimum dla stabilności
noise_amp	0.4	Amplituda fluktuacji termicznych (szumu)
draw_every	1	Co ile kroków odświeżyć wizualizację

Tabela 2: Parametry klasy `SimConfig`.

4.3. Kluczowe fragmenty kodu

Inicjalizacja stanu (engine.py, klasa `SimState`):

Listing 1: Inicjalizacja pola m i siatki Fouriera.

```

1 class SimState:
2     def __init__(self, cfg: SimConfig):
3         self.cfg = cfg
4         amp = min(cfg.noise_amp, min(1.0 - cfg.m0, cfg.m0 + 1.0))
5         noise = np.random.uniform(-amp, amp, size=(cfg.N, cfg.N))
6         self.m = cfg.m0 + noise # warunek poczatkowy
7
8         self.DX = cfg.L / cfg.N
9         k = np.fft.fftfreq(cfg.N, d=self.DX) * 2 * np.pi
10        self.eps = cfg.alpha * self.DX # szerokosc granicy faz
11
12        self.kx, self.ky = np.meshgrid(k, k, indexing='ij')
13
14        # maska dealiasingowa (regula 2/3)
15        k_max = np.pi / self.DX
16        k_cutoff = (2.0 / 3.0) * k_max
17        k_mag = np.sqrt(self.kx**2 + self.ky**2)
18        self.dealias_mask = np.where(k_mag < k_cutoff, 1.0, 0.0)

```

Rdzeń solvera (engine.py, klasa Solver):

Listing 2: Krok czasowy solvera C-H (SISM).

```

1 @njit(cache=True)
2 def calculate_nonlin(m):
3     return m**3 - m # czlon jawny (explicit)
4
5 @njit(cache=True)
6 def calculate_new_m(mk, fk, kx, ky, dt, eps):
7     k2 = kx**2 + ky**2
8     num = mk - dt * k2 * fk
9     denom = 1 + dt * (eps**2) * (k2**2)
10    return num / denom # schemat pol-niejawny (eq. 3.6)
11
12 class Solver:
13     def step(self, state: SimState, cfg: SimConfig):
14         f = calculate_nonlin(state.m)
15         fk = np.fft.fft2(f) * state.dealias_mask # + dealiasing
16         mk = np.fft.fft2(state.m)
17
18         mk_new = calculate_new_m(mk, fk, state.kx, state.ky,
19                                 cfg.DT, state.eps)
20         state.m = np.real(np.fft.ifft2(mk_new))

```


4.4. Obliczanie energii

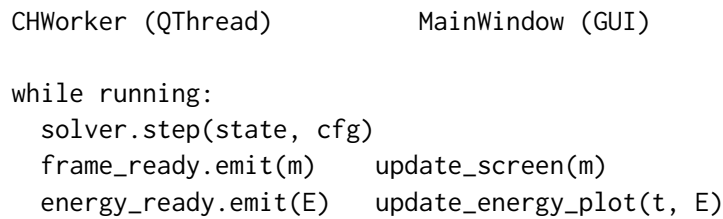
Energia liczona jest numerycznie według wzoru (2.13) z użyciem `np.gradient` do gradientów przestrzennych:

Listing 3: Obliczanie energii swobodnej.

```
1 def calculate_energy(self):
2     nonlin = 0.25 * (self.m**2 - 1)**2
3     grad_x, grad_y = np.gradient(self.m, self.DX)
4     lin = 0.5 * self.eps**2 * (grad_x**2 + grad_y**2)
5     return np.sum((nonlin + lin) * self.DX**2)
```

4.5. Architektura wielowątkowa

Obliczenia uruchomione są w osobnym wątku `CHWorker` dziedziczącym po `QThread`. Sygnały Qt (`frame_ready`, `energy_ready`) przesyłają dane do GUI bez blokowania pętli zdarzeń. Schemat przepływu danych przedstawia rysunek 1.



Rysunek 1: Schemat komunikacji między wątkiem obliczeniowym a GUI.

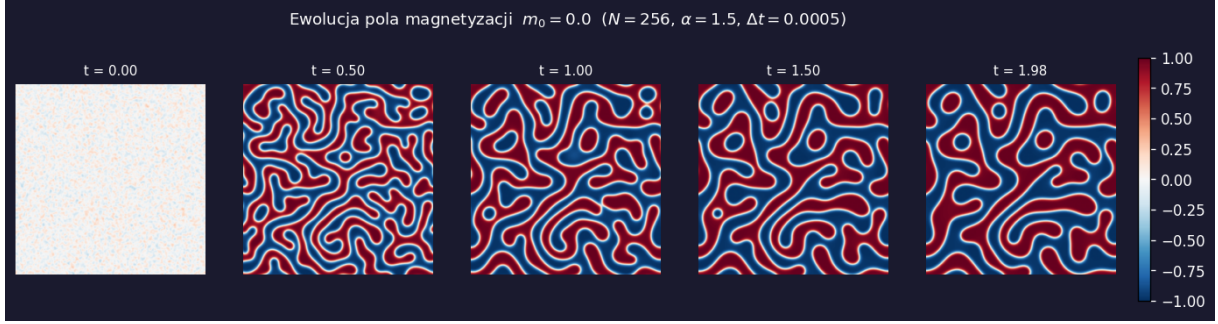
4.6. Akceleracja z Numbą

Funkcje `calculate_nonlin` i `calculate_new_m` skompilowane są przez dekorator `@jit(cache=True)` (Numba Just-In-Time). Eliminuje to narzut interpretera Pythona i pozwala kompilatorowi LLVM na wektoryzację operacji elementarnych, co daje przyspieszenie $\sim 5\text{--}10\times$ względem czystego NumPy dla dużych siatek.

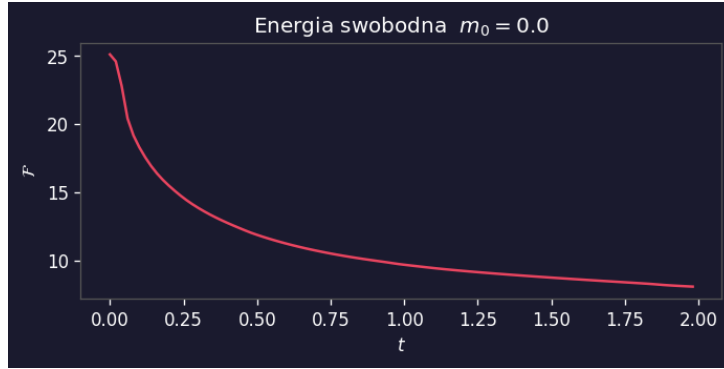
5. Wyniki

Poniżej przedstawiono wyniki trzech scenariuszy symulacyjnych wyróżniających się wartością średniej magnetyzacji m_0 . We wszystkich przypadkach: $N = 256$, $L = 10$, $\Delta t = 5 \times 10^{-4}$, $\alpha = 1.5$, $\text{noise_amp} = 0.4$.

5.1. Przypadek $m_0 = 0$: struktury labiryntowe



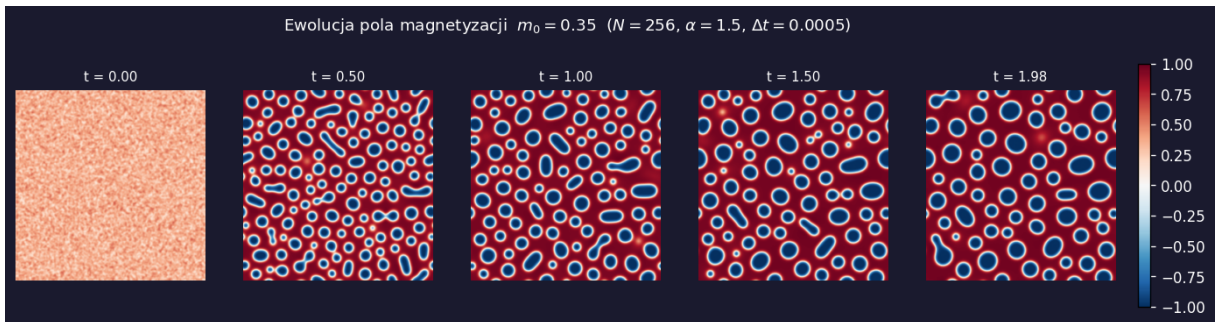
Rysunek 2: Ewolucja pola $m(\mathbf{r}, t)$ dla $m_0 = 0$. Kolor czerwony: $m \approx +1$, niebieski: $m \approx -1$. Widoczne charakterystyczne struktury labiryntowe (rozkład spinodalny).



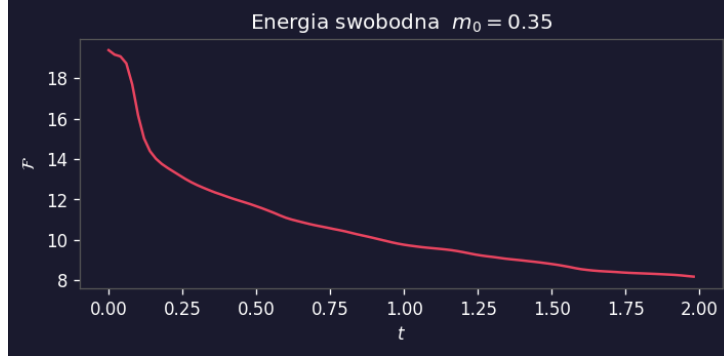
Rysunek 3: Spadek energii swobodnej $\mathcal{F}(t)$ dla $m_0 = 0$.

Przy zerowej średniej magnetyzacji obie fazy ($m = +1$ i $m = -1$) są równorzędne. Szybka separacja spinodalna prowadzi do tworzenia wzajemnie przenikających się, labiryntowych domen. Energia spada gwałtownie w fazie rozkładu spinodalnego, a następnie znacznie wolniej podczas koalescencji (grubienia domen).

5.2. Przypadek $m_0 = +0.35$: krople mniejszościowe



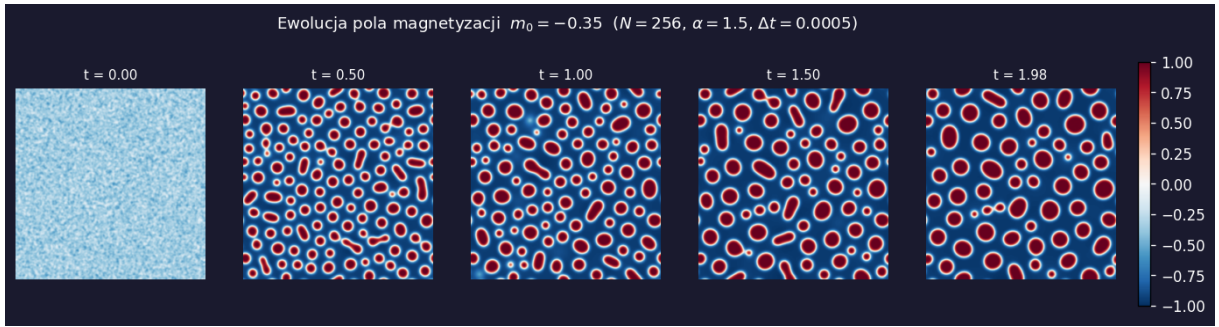
Rysunek 4: Ewolucja pola $m(\mathbf{r}, t)$ dla $m_0 = +0.35$. Faza $m \approx -1$ (niebieskie krople) rozproszona w macierzy $m \approx +1$ (tło czerwone).



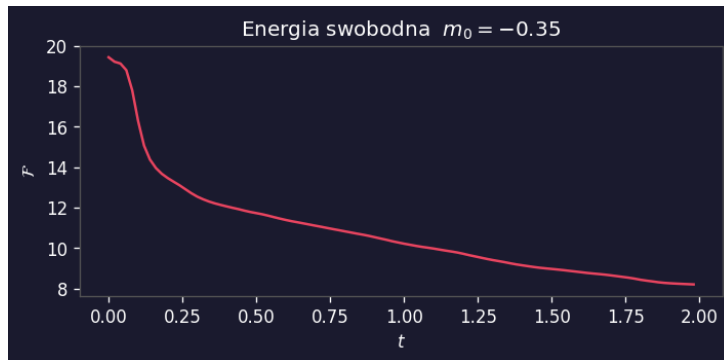
Rysunek 5: Spadek energii swobodnej $\mathcal{F}(t)$ dla $m_0 = +0.35$.

Dodatnie przesunięcie $m_0 > 0$ faworyzuje fazę $m \approx +1$ jako fazę ciągłą. Mniejszościowa faza $m \approx -1$ formuje izolowane krople, które stopniowo koalescuja.

5.3. Przypadek $m_0 = -0.35$: krople odwrotne



Rysunek 6: Ewolucja pola $m(\mathbf{r}, t)$ dla $m_0 = -0.35$. Faza $m \approx +1$ (czerwone krople) w macierzy $m \approx -1$ (tło niebieskie).



Rysunek 7: Spadek energii swobodnej $\mathcal{F}(t)$ dla $m_0 = -0.35$.

Przypadek $m_0 = -0.35$ jest symetryczny do poprzedniego z zamienionymi rolami faz. Struktura kropelkowa jest analogiczna, lecz w odwróconych kolorach, co potwierdza symetrię $m \rightarrow -m$ równania (2.11).

5.4. Weryfikacja zachowania energii

We wszystkich trzech przypadkach zaobserwowano:

1. **Monotonicznie malejącą** energię $\mathcal{F}(t) \geq 0$ – zgodnie z drugą zasadą termodynamiki (układy dyssypatywne).
2. **Dwa reżimy** dynamiki energii:
 - faza szybkiej separacji spinodalnej (gwałtowny spadek),
 - faza powolnej koalescencji (energia plateau-like, malejąca)
3. **Zachowanie magnetyzacji** $\int m \, d\mathbf{r} = \text{const}$ – równanie C–H ma postać zachowawczą ($\partial_t m = -\nabla \cdot \mathbf{J}$).

6. Podsumowanie

W ramach projektu z powodzeniem zaimplementowano numeryczny solver równania Cahn–Hilliarda w 2D z wykorzystaniem:

- **Pół-niejawnej metody spektralnej (SISM)**, która łączy efektywność FFT z warunkowo stabilnym schematem dla sztywnych członów liniowych;
- **Reguły dealiasingowej 2/3**, eliminującej błędy aliasingu generowane przez człon nieliniowy m^3 ;
- **Kompilatora Numba NJIT**, który wielokrotnie przyspiesza obliczenia względem czystego NumPy;
- **Wielowątkowego GUI** (PySide6 + pyqtgraph), umożliwiającego wizualizację symulacji w czasie rzeczywistym.

Wyniki symulacji jakościowo odtwarzają znane zjawiska: struktury labiryntowe ($m_0 = 0$), układ kropelkowy ($m_0 \neq 0$) oraz monotonicznie malejącą energię swobodną.

W repozytorium na githubie <https://github.com/sosna77/cahn-hilliard> znajduje się kod odpowiedzialny za generowanie symulacji. Instrukcja do uruchomienia symulacji po ustawieniu jej parametrów znajduje się w pliku `README.md`. Kod użyty do wygenerowania grafik do raportu znajduje się w `src/generate_plots.py`, a grafiki razem z gifami odpowiadającymi scenariuszom ($m_0 = 0$, $m_0 = +0.35$, oraz $m_0 = -0.35$) znajdują się w katalogu `output`.

Literatura

- [1] J. W. Cahn, J. E. Hilliard, *Free Energy of a Nonuniform System. I. Interfacial Free Energy*, J. Chem. Phys. **28**, 258 (1958).